Fundamentals of Cybersecurity

Mohammad Hosseini

Faculty of Computer Science and Engineering
Shahid Beheshti University

Security in Operating Systems



Operating system

An **operating system** (**OS**) is the core software layer that manages hardware, software, and resources in a computer system.

Key Functions of an OS:

- Hardware abstraction
- Resource management
- Process management
- File management
- User interface



Operating system security

- The OS is **the first line of defense.** it must ensure that:
  - Only authorized users and processes can access resources.
  - One user is protected from another, and critical areas of memory or storage are not overwritten by unauthorized processes.
  - Sensitive data is protected from malicious actors.
  - System integrity is maintained.
- All security mechanisms provided for programs or networks depend on the security of their underlying infrastructure, namely OS.
  - If OS is compromised, the entire system is at risk.
- Operating system security refers to the measures and mechanisms designed to protect the OS from threats, such as unauthorized access, malware, or exploitation of vulnerabilities.

Operating system security

Operating system security is a crucial area of focus in cybersecurity.

- It **enforces security** for all users and applications.
- The OS **itself** must be **protected** from being exploited.

Security enforcement by OS

How does an OS enforce security?

- When the operating system initializes at system boot time, it initiates tasks in an orderly sequence:
  - primitive functions and device drivers first, then process controllers, followed by file and memory management routines, and finally the user interface.
  - To establish **security**, early tasks establish a firm defense to constrain later tasks.

How does an OS enforce security?

- Authentication
  - The OS verifies user identities through mechanisms like passwords or biometrics.
- Access control
  - The OS governs who can access files, processes, and other resources.
  - It ensures that users and applications operate with the **least privilege** required to accomplish their tasks.
- Process isolation
  - The OS keeps processes separate from each other to prevent one process from interfering with or spying on another.
  - Protection against malicious processes.
  - Containment of errors and crashes.
  - Support for multi-user environments.
- System integrity
  - Preventing tampering at the firmware level.

Operating system security

What happens if an attacker compromises the OS?

Separation of privileges

It ensures that no single user or process has unchecked power over the system.

Most operating systems, like Linux, Windows, and macOS, operate in at least **two distinct modes**:

- User mode, where applications and users operate. They have limited access to system resources.
- Kernel mode, where the operating system core functions run. This mode has unrestricted access to all hardware and memory.
  - Security enforcement mechanisms of an OS run in kernel mode to prevent them from being bypassed by user applications.

Separation of privileges

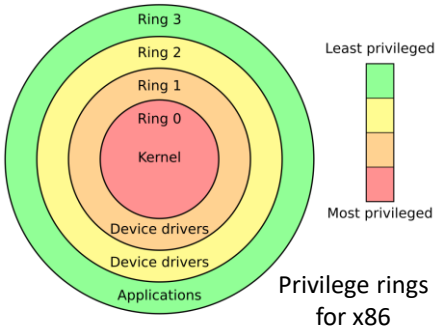
- How does OS enforce security in user mode?
- What prevents user-mode applications from interfering with the system or other applications?
  - Many operating system protection mechanisms have been supported by hardware.

Privilege levels in CPU

- Modern processors operate with multiple privilege levels, often referred to as **rings**.

➢ Ring 0 (**Kernel Mode**): The most privileged level where the OS kernel runs. It has direct access to hardware and can execute privileged instructions.

➢ Ring 3 (**User Mode**): The least privileged level where applications run. They cannot execute privileged instructions or access hardware directly.



Privilege rings for x86

- Applications are restricted to Ring 3, and any attempt to execute privileged instructions (e.g., directly interacting with hardware) results in a trap or exception from the CPU, which the OS handles as an error.

Privilege switching

- At system startup, the CPU begins execution in the most privileged level (Ring 0), where the OS kernel resides.
- Before running an application, the OS sets the CPU to Ring 3 by configuring the CPU's privilege level in the **Control Registers**.
- An application running in user mode cannot directly change the privilege level of the CPU to gain kernel mode privileges.
  - ➢ If an application in user mode attempts to execute an instruction that modifies the CPU privilege level (e.g., by modifying control registers or using privileged instructions), the CPU generates a trap or fault that is caught by the operating system. The OS then terminates the offending process or logs an error.
- How can an application access hardware?!

System calls as the gateway

- To interact with hardware, applications must make system calls to the OS, which operates in kernel mode.
  - Example: When an application wants to write to a file, it calls a function like `write()`. This function is implemented as a system call, which temporarily switches the CPU to kernel mode to interact with the hardware (e.g., disk).
- This ensures that the OS validates and controls all hardware interactions, preventing unauthorized access.
  - A user-level application cannot directly modify hardware registers or read another application's data.

Memory protection

- The OS manages **virtual memory** using a memory management unit (MMU) in the CPU. Each application is assigned its own virtual address space, which isolates it from:
  - Other applications' memory.
  - The kernel's memory (including hardware device drivers).
- The hardware's memory-mapped regions (e.g., device registers) are only accessible in **kernel mode**.
  - If an application tries to access these addresses, the MMU blocks the request and triggers a protection fault.

Access control

Access control ensures that only authorized users or processes can access specific resources, whether it's files, devices, or system settings.

Types of access control:

- Discretionary Access Control (DAC)
- Mandatory Access Control (MAC)
- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC)
- Context-Based Access Control (CBAC)
- …

Access control

- Discretionary Access Control
  - A subject with a certain access permission is capable of passing that permission (perhaps indirectly) on to any other subject.
  - Resource owners decide who gets access.

```
jamie@debian:~$ ls -l
total 19208
-rwxr-xr-x 1 root  root   4703728 Dec 17 07:01 Battle.net-Setup.exe
drwxr-xr-x 3 jamie jamie     4096 Nov  5 03:30 Desktop
drwxr-xr-x 2 jamie jamie     4096 Jun  5  2018 Documents
drwxr-xr-x 3 jamie jamie     4096 Dec 17 06:49 Downloads
-rw-r--r-- 1 jamie jamie   179765 Dec 17 07:01 Linux_for_beginners.pdf
-rw-r--r-- 1 jamie jamie   458980 Dec 17 06:59 metamorphose2_0.8.2-1_all.deb
drwxr-xr-x 2 jamie jamie     4096 Apr 30  2018 Music
-rw-r--r-- 1 jamie jamie     1520 Dec 17 07:01 Neofetch
-rw-r--r-- 1 root  root  13902480 Dec 17 07:01 pdfsam_3.3.6-1_all.deb
-rw------- 1 root  root    375728 Dec 17 07:01 PDFsam_merge.pdf
drwxr-xr-x 2 jamie jamie     4096 Apr 30  2018 Pictures
drwxr-xr-x 2 jamie jamie     4096 Apr 30  2018 Public
drwxr-xr-x 2 jamie jamie     4096 Apr 30  2018 Templates
drwxr-xr-x 2 jamie jamie     4096 Apr 30  2018 Videos
```

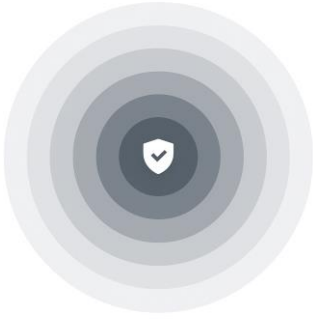
  - Weakness: The downside of DAC is that users might unintentionally assign broad permissions, leading to vulnerabilities.

Access control

- Mandatory Access Control
  - The security policy is centrally controlled by a policy administrator and is guaranteed to be enforced for all users.
  - Users cannot override the policy and, for example, grant access to files that would otherwise be restricted.
- Role-Based Access Control
  - It simplifies privilege management by assigning permissions to roles, rather than individual users.

Defense in depth

- This principle emphasizes layering multiple security measures so that if one fails, others still provide protection.
- In the context of OS security, Defense in Depth includes:
  - User Authentication: Ensuring that only authorized users can access the system.
  - Access Controls: Limiting what users and processes can do.
  - Memory Protection Mechanisms: Preventing unauthorized access to memory.
  - Logging and Monitoring: Detecting suspicious activity in real time.



Protecting OS

Malware

- Malware can target operating systems to gain unauthorized access, steal data, or disrupt functionality.
  - Virus, worm, trojan, rootkit
- We have previously talked about solutions to mitigate malware threats.
  - One other solution: Sandboxing

Sandbox

- Sandboxing is a security mechanism that isolates applications or processes in a restricted and controlled environment.
- The goal is to prevent untrusted or potentially malicious code from affecting the rest of the system or accessing sensitive data.
- It prevents the application-under-test from interacting with the host system or other processes.
- It is used to test unknown or suspicious software in a safe environment.



honeypot

- A honeypot is a decoy system, network, or resource that is designed to attract attackers.
- It mimics a legitimate target, such as a server, database, or other infrastructure, to lure cybercriminals or malicious actors.
- It can serve as a distraction to keep attackers away from actual systems.
- It helps security teams understand the motivations and methods of cybercriminals.



Rootkit

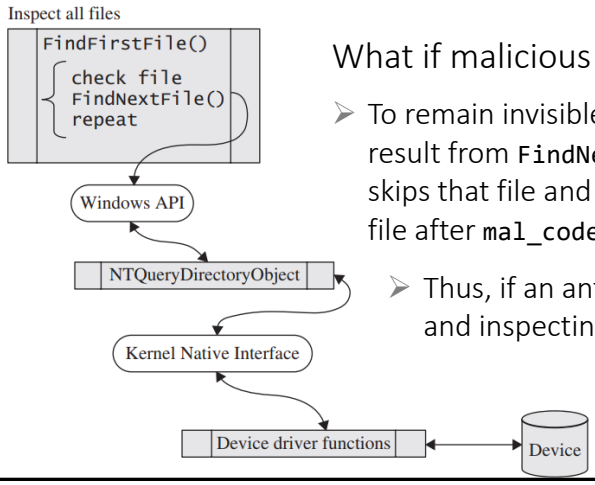
- Put yourself in the mind of an attacker.
  - If you want persistency, you want an attack that is really difficult to detect so your victim cannot find and try to eradicate your code.
- Two conditions can help you remain undiscovered:
  - Executing your code before other programs that might block your execution.
  - Preventing from being detected as a file or process.
- Being in control early in the system boot cycle would allow you to control the other system defenses instead of their controlling you.
  - If your code is introduced early enough and with privileges of root, it can override other normal system functions that would detect its presence.

Rootkit

- Rootkit: a type of malware designed to gain unauthorized access to a computer system (privileges of root) and hide its presence.
- Rootkits often operate at the deepest levels of a system—such as the operating system kernel or even lower—to maintain persistent control over the system while avoiding detection by security tools.
- A rootkit effectively becomes part of the operating system kernel.

How a rootkit can hide itself

- Antivirus tools (and most programs) do not contain code to query the disk and identify files. Instead, the tools call built-in functions through an API to get this information.



What if malicious code intruded on that sequence of calls?

- To remain invisible, the rootkit intercepts these calls so that if the result from `FindNextFile()` points to `mal_code.exe`, the rootkit skips that file and executes `FindNextFile()` again to find the next file after `mal_code.exe`.
  - Thus, if an antivirus tool is scanning by obtaining a list of files and inspecting each one, the tool will miss the malicious file.

Types of rootkits

- User mode
  - Operate at the user level (runs with only the permissions of the ordinary user).
  - Easier to detect compared to kernel-mode rootkits.
- Kernel mode
  - Operate at the kernel level (operating system core).
  - Harder to detect and remove
- Bootkit
  - Infect the bootloader or operating system startup process.
  - Load before the operating system, allowing them to control the system from the very beginning.
  - Example: Attacks on the Master Boot Record (MBR).
- Firmware
  - Reside in firmware, such as the BIOS, UEFI

Non-malicious rootkits

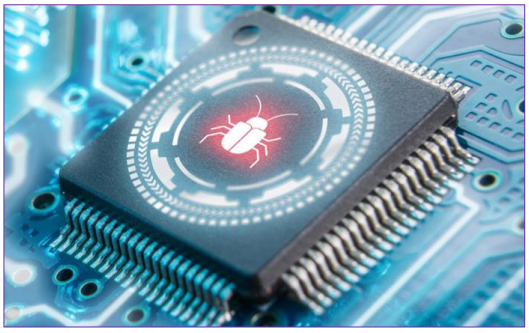
Not every rootkit is malicious.

- Parental control or employee monitoring
  - e.g., eBlaster, Spector
- Anti-cheat systems in gaming
  - e.g., BattlEye, XIGNCODE, Easy Anti-Cheat
- Digital Rights Management (DRM)
- Security and monitoring Tools

Bootloader and BIOS/UEFI Threats

- Bootloader: a small program that loads the operating system into memory and starts it. (e.g., GRUB, used in Linux)
- BIOS/UEFI: firmware embedded in a chip on the motherboard. It initializes hardware during the boot process and hands control to the bootloader.

- A compromised bootloader or BIOS can load malicious code into memory or disable OS security features.
- Malware or rootkits injected into BIOS/UEFI or the bootloader can **persist** even after reformatting the hard drive or reinstalling the OS.



Bootloader and BIOS/UEFI Threats

How can an attacker modify BIOS or UEFI? Is it possible?

- BIOS and UEFI firmware are stored in non-volatile memory chips on the motherboard.
  - These chips are write-enabled during firmware updates to allow manufacturers to push updates for bug fixes, compatibility improvements, or security patches.
- To **update** the firmware, the system needs to temporarily allow write access to the chip.
  - Malware can exploit operating system vulnerabilities to gain administrator (root) or kernel-level privileges.
- While legitimate updates typically use signed firmware and controlled processes, malicious actors can exploit vulnerabilities in the update mechanism to inject unauthorized changes.

Bootloader and BIOS/UEFI Threats

- UEFI Secure Boot is a verification mechanism for ensuring that code launched by a computer's UEFI firmware is trusted.
- It is designed to protect a system against malicious code, such as rootkits, being loaded and executed early in the boot process, before the operating system has been loaded.
- It prevents the loading of UEFI drivers or OS bootloaders (such as GRUB or Windows Boot Manager) that are not **signed** with an acceptable digital signature.